Module End Assignment

Analyzing E-Learning Platform Purchases using MySQL

1. Database Setup & Data Entry

- Create a new database.

```sql
-- Analyzing E-Learning Platform Purchases using MySQL

create database onlinelearner_db;

use onlinelearner_db;
```

- Create the three tables with proper:

  - Primary keys, Foreign keys, Appropriate data types

```sql
create table learners(
    learner_id int primary key,
    full_name varchar(50),
    country varchar(50)
);

create table courses(
    course_id int primary key,
    course_name varchar(50),
    category varchar(30),
    unit_price int
);

create table purchases(
    purchase_id int primary key,
    learner_id int,
    course_id int,
    quantity int,
    purchase_date date,

Foreign Key(learner_id) References learners(learner_id),
Foreign Key(course_id) References courses(course_id)
);
```

- Insert sample data:

  - 4–5 learners

  - 4–5 courses (different categories)

  - 6–8 purchase records

```sql
Insert into learners (learner_id, full_name, country) values
('1001', 'Gayathri', 'India'),
('1002', 'Jing', 'China'),
('1003', 'Himari', 'Japan'),
('1004', 'Maria', 'Russia'),
('1005', 'Amara', 'Africa');

Insert into courses (course_id, course_name, category, unit_price) values
('101', 'Chartered Accountant', 'Finance', '10000.00'),
('102', 'Chinese for Beginners', 'Language', '4000.00'),
('103', 'Global Health Policy', 'Medicine', '15000.00'),
('104', 'Master of Data Science', 'Computer Science', '20000.00'),
('105', 'Beauty Therapy', 'Cosmetology', '8000.00');

Insert into purchases (purchase_id, learner_id, course_id, quantity, purchase_date) values
('201', '1001', '101', 1, '2026-08-01'),
('202', '1002', '102', 2, '2026-08-03'),
('203', '1003', '103', 1, '2026-08-05'),
('204', '1004', '104', 1, '2026-08-07'),
('205', '1005', '105', 3, '2026-08-10'),
('206', '1001', '104', 1, '2026-08-12'),
('207', '1002', '103', 2, '2026-08-15'),
('208', '1003', '102', 1, '2026-08-18');
```

```sql
Select * from courses;
Select * from learners;
Select * from purchases;
```

Result Grid | Filter Rows: | Edit: | Export

course_id	course_name	category	unit_price
101	Chartered Accountant	Finance	10000
102	Chinese for Beginners	Language	4000
103	Global Health Policy	Medicine	15000
104	Master of Data Science	Computer Science	20000
105	Beauty Therapy	Cosmetology	8000
NULL	NULL	NULL	NULL

```sql
Select * from courses;
Select * from learners;
Select * from purchases;
```

Result Grid | Filter Rows: | Edit: | Export

learner_id	full_name	country
1001	Gayathri	India
1002	Jing	China
1003	Himari	Japan
1004	Maria	Russia
1005	Amara	Africa
NULL	NULL	NULL

```
54
55 •  Select * from courses;
56 •  Select * from learners;
57 •  Select * from purchases;
58
```

Result Grid | Filter Rows: | Edit: | Export/Import:

purchase_id	learner_id	course_id	quantity	purchase_date	Total_amount
201	1001	101	1	2026-08-01	10000.00
202	1002	102	2	2026-08-03	8000.00
203	1003	103	1	2026-08-05	15000.00
204	1004	104	1	2026-08-07	20000.00
205	1005	105	3	2026-08-10	24000.00
206	1001	104	1	2026-08-12	20000.00
207	1002	103	2	2026-08-15	30000.00
208	1003	102	1	2026-08-18	4000.00
NULL	NULL	NULL	NULL	NULL	NULL

2. Data Exploration Using Joins

Use INNER, LEFT, and RIGHT JOIN to:

● Combine learner, course, and purchase data

● *Display:* Learner name, Course name, Category, Quantity, Total amount, Purchase

date

● Presentation Requirements:

■ Format currency to 2 decimal places

■ Use column aliases

■ Sort by the highest total amount

```
58
59 •  Alter table purchases
60    Add column Total_amount decimal(10,2);
61
62 •  Update purchases as p
63    Inner Join courses as c
64        on p.course_id = c.course_id
65    Set p.total_amount = p.quantity * c.unit_price;
```

Inner Join:

```
66
67 •  Select l.full_name as "Learner Name",
68           c.course_name as "Course Name",
69           c.category as "Category",
70           p.quantity as "Quantity",
71           p.purchase_date as "Purchase Date",
72           p.total_amount as "Total Amount"
73    From purchases p
74    Inner Join learners l
75        on p.learner_id = l.learner_id
76    Inner Join courses c
77        on p.course_id = c.course_id
78    Order by p.total_amount desc;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Learner Name	Course Name	Category	Quantity	Purchase Date	Total Amount
Jing	Global Health Policy	Medicine	2	2026-08-15	30000.00
Amara	Beauty Therapy	Cosmetology	3	2026-08-10	24000.00
Gayathri	Master of Data Science	Computer Science	1	2026-08-12	20000.00
Maria	Master of Data Science	Computer Science	1	2026-08-07	20000.00
Himari	Global Health Policy	Medicine	1	2026-08-05	15000.00
Gayathri	Chartered Accountant	Finance	1	2026-08-01	10000.00
Jing	Chinese for Beginners	Language	2	2026-08-03	8000.00
Himari	Chinese for Beginners	Language	1	2026-08-18	4000.00

Left Join:

```
79
80 •  Select l.full_name as "Learner Name",
81           c.course_name as "Course Name",
82           c.category as "Category",
83           p.quantity as "Quantity",
84           p.purchase_date as "Purchase Date",
85           p.total_amount as "Total Amount"
86    From learners l
87    Left Join purchases p
88        on l.learner_id = p.learner_id
89    Left Join courses c
90        on p.course_id = c.course_id
91    Order by p.total_amount desc;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Learner Name	Course Name	Category	Quantity	Purchase Date	Total Amount
Jing	Global Health Policy	Medicine	2	2026-08-15	30000.00
Amara	Beauty Therapy	Cosmetology	3	2026-08-10	24000.00
Gayathri	Master of Data Science	Computer Science	1	2026-08-12	20000.00
Maria	Master of Data Science	Computer Science	1	2026-08-07	20000.00
Himari	Global Health Policy	Medicine	1	2026-08-05	15000.00
Gayathri	Chartered Accountant	Finance	1	2026-08-01	10000.00
Jing	Chinese for Beginners	Language	2	2026-08-03	8000.00
Himari	Chinese for Beginners	Language	1	2026-08-18	4000.00

Right Join:

```
93 •  Select l.full_name as "Learner Name",
94           c.course_name as "Course Name",
95           c.category as "Category",
96           p.quantity as "Quantity",
97           p.purchase_date as "Purchase Date",
98           p.total_amount as "Total Amount"
99    From purchases p
100   Right Join courses c
101       on p.course_id = c.course_id
102   Left Join learners l
103       on l.learner_id = p.learner_id
104   Order by p.total_amount desc;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Learner Name	Course Name	Category	Quantity	Purchase Date	Total Amount
Jing	Global Health Policy	Medicine	2	2026-08-15	30000.00
Amara	Beauty Therapy	Cosmetology	3	2026-08-10	24000.00
Maria	Master of Data Science	Computer Science	1	2026-08-07	20000.00
Gayathri	Master of Data Science	Computer Science	1	2026-08-12	20000.00
Himari	Global Health Policy	Medicine	1	2026-08-05	15000.00
Gayathri	Chartered Accountant	Finance	1	2026-08-01	10000.00
Jing	Chinese for Beginners	Language	2	2026-08-03	8000.00
Himari	Chinese for Beginners	Language	1	2026-08-18	4000.00

3. Core Analytical Queries (Q1–Q5)

Q1. Display each learner's total spending with their country.

```
105
106 •  Select l.full_name as "Learner Name",
107          l.country as "Country",
108          sum(p.Total_amount) as "Total Amount"
109   From learners l
110   Left Join purchases p
111       on l.learner_id = p.learner_id
112   Group by l.learner_id,
113            l.full_name,
114            l.country
115   Order by sum(p.Total_amount) desc;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Learner Name	Country	Total Amount
Jing	China	38000.00
Gayathri	India	30000.00
Amara	Africa	24000.00
Maria	Russia	20000.00
Himari	Japan	19000.00

Q2. Find the top 3 most purchased courses by quantity.

```
116
117 •  Select c.course_name as "Course Name",
118           Sum(p.quantity) as "Total Quantity Purchased"
119    from Courses c
120    Inner Join purchases p
121        on c.course_id = p.course_id
122    Group by c.course_id,
123             c.course_name
124    Order by Sum(p.quantity) desc
125    Limit 3;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

Course Name	Total Quantity Purchased
Beauty Therapy	3
Chinese for Beginners	3
Global Health Policy	3

Q3. Show each category's:

- Total revenue
- Number of unique learners

```
126
127 •  Select c.category as "Category",
128           sum(p.total_amount) as "Total Revenue",
129           count(distinct p.learner_id) as "Unique Learners"
130    From courses c
131    Left Join purchases p
132        on c.course_id = p.course_id
133    Group By c.category
134    Order by sum(p.total_amount) desc;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Category	Total Revenue	Unique Learners
Medicine	45000.00	2
Computer Science	40000.00	2
Cosmetology	24000.00	1
Language	12000.00	2
Finance	10000.00	1

Q4. List learners who purchased from more than one category.

```
135
136 •  Select l.full_name as "Learners",
137           count(distinct c.category) as "Number of Category"
138    From learners l
139    Inner join purchases p
140        on l.learner_id = p.learner_id
141    Inner Join courses c
142        on p.course_id = c.course_id
143    Group by l.learner_id,
144             l.full_name
145    Having Count(distinct c.category) > 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Learners	Number of Category
Gayathri	2
Jing	2
Himari	2

Q5. Identify courses never purchased.

```
146
147 •  Select c.course_id as "Course ID",
148           c.course_name as "Course Name",
149           c.category as "Category"
150    From courses c
151    Left join purchases p
152        on c.course_id = p.course_id
153    where p.course_id is null;
154
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Course ID	Course Name	Category

4. Subqueries & Correlated Subqueries

Q6. Find learners whose total spending is above the average learner spending.

```
157 •  Select l.full_name as "Learners",
158           l.country as "Country",
159           sum(p.Total_amount) as "Total Spend"
160    From learners l
161    Left Join purchases p
162        on l.learner_id = p.learner_id
163    Group By l.full_name,
164             l.country
165    Having sum(p.Total_amount) >
166        (Select avg(learner_total)
167            from (Select learner_id,
168                         Sum(Total_amount) as learner_total
169                         from purchases
170                         Group By learner_id
171                ) as spending
172        )
173    Order by sum(p.Total_amount) desc;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Learners	Country	Total Spend
Jing	China	38000.00
Gayathri	India	30000.00

<u>Note:</u> Initially, the course category was defined as field of course like Mathematics, Commerce etc. Since Q7 specifically mentions about the category as Beginner, I have performed following changes in the table 'Courses' where the new column is named as 'Level'.

```
174
175 •  Alter table courses
176    Add column level varchar(20);
177
178 •  Update courses
179    set level = "Intermediate"
180    where course_id = "101";
181
182 •  Update courses
183    set level = "Beginner"
184    where course_id = "102";
185
186 •  Update courses
187    set level = "Intermediate"
188    where course_id = "103";
189
190 •  Update courses
191    set level = "Advanced"
192    where course_id = "104";
193
194 •  Update courses
195    set level = "Intermediate"
196    where course_id = "105";
```

```
197
198 •  Select * from Courses;
199
200
201
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

course_id	course_name	category	unit_price	level
101	Chartered Accountant	Finance	10000	Intermediate
102	Chinese for Beginners	Language	4000	Beginner
103	Global Health Policy	Medicine	15000	Intermediate
104	Master of Data Science	Computer Science	20000	Advanced
105	Beauty Therapy	Cosmetology	8000	Intermediate
NULL	NULL	NULL	NULL	NULL

Q7. Display courses whose price is higher than any course in the ‘Beginner’ category.

```
199
200 •  Select course_id as "Course ID",
201           course_name as "Course Name",
202           unit_price as "Price",
203           level as "Course Level"
204    from courses
205    where unit_price > Any
206    (Select unit_price
207        from courses
208        where level = "Beginner"
209    )
210    Order by unit_price desc;
211
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Course ID	Course Name	Price	Course Level
104	Master of Data Science	20000	Advanced
103	Global Health Policy	15000	Intermediate
101	Chartered Accountant	10000	Intermediate
105	Beauty Therapy	8000	Intermediate

Q8 . Find learners who spent more than the average spending in their country

```
Select l.full_name as "Learners",
       l.country as "Country",
       sum(p.Total_amount) as "Total Spend"
From learners l
Inner Join purchases p
    on l.learner_id = p.learner_id
Group By l.learner_id,
         l.full_name,
         l.country
Having sum(p.Total_amount) >
    (Select avg(learner_total)
        from (Select l2.learner_id,
                     l2.country,
                     Sum(p2.Total_amount) as learner_total
              from learners l2
              Inner Join purchases p2
              on l2.learner_id = p2.learner_id
              where l2.country = l.country
              Group By l2.learner_id,
                       l2.country
             ) as country_spending
     )
Order by l.country, sum(p.Total_amount) desc;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Learners	Country	Total Spend

This returns no rows since each country has only one learner and their spending is equal to their country average.

5. CTE, CASE, View, and NULL Handling

Q9. Use a CTE to calculate total spending per learner, then:

Display learners with spending above 10,000.

```sql
With learner_spending as
(Select l.learner_id,
        l.full_name,
        l.country,
        sum(p.Total_amount) as "Total_Spend"
from Learners l
Inner Join Purchases p
    on l.learner_id = p.learner_id
Group by l.learner_id,
         l.full_name,
         l.country
)
Select learner_id as "Learner ID",
       full_name as "Learner Name",
       country as "Country",
       Total_spend as "Total_spending"
from learner_spending
where Total_spend > 10000
Order by Total_spend desc;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Learner ID	Learner Name	Country	Total_spending
1002	Jing	China	38000.00
1001	Gayathri	India	30000.00
1005	Amara	Africa	24000.00
1004	Maria	Russia	20000.00
1003	Himari	Japan	19000.00

Q10. CASE Expression

Classify learners based on spending:

- Above 15,000 → “High Value”,
- 8,000–15,000 → “Medium Value”,
- Below 8,000 → “Low Value”

```sql
Select l.full_name as "Learners Name",
       l.country as "Country",
       sum(p.Total_amount) as "Total Spend",
Case
    when sum(p.Total_amount) > 15000
        then "High Value"
    when sum(p.Total_amount) between 8000 and 15000
        then "Medium Value"
    Else "Low Value"
End as "Learner Category"
from Learners l
Inner Join Purchases p
    on l.learner_id = p.learner_id
Group by l.learner_id,
         l.full_name,
         l.country
Order by sum(p.Total_amount) desc;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Learners Name	Country	Total Spend	Learner Category
Jing	China	38000.00	High Value
Gayathri	India	30000.00	High Value
Amara	Africa	24000.00	High Value
Maria	Russia	20000.00	High Value
Himari	Japan	19000.00	High Value

Q11 . NULL Handling

● Display all courses and replace NULL purchase counts with 0 using: IFNULL() or COALESCE()

```
271
272 •  Select c.course_id as "Course ID",
273           c.course_name as "Course Name",
274           c.category as "Category",
275           Ifnull(count(p.purchase_id),0) as "Purchase Count"
276    from courses c
277    Left Join purchases p
278    on c.course_id = p.course_id
279    Group by c.course_id,
280           c.course_name,
281           c.category
282    Order by "Purchase Count" desc;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Course ID	Course Name	Category	Purchase Count
101	Chartered Accountant	Finance	1
102	Chinese for Beginners	Language	2
103	Global Health Policy	Medicine	2
104	Master of Data Science	Computer Science	2
105	Beauty Therapy	Cosmetology	1

Q12 . View

- **Create a view: category_performance_view**
- **Showing:**
- *Category*
- *Total revenue*
- *Number of purchases*
- *Average revenue per purchase*

```sql
Create view category_performance_view as
Select c.category as "Category",
       sum(p.Total_amount) as "Total Revenue",
       count(p.purchase_id) as "Number of Purchases",
       Round(sum(p.Total_amount)/count(p.purchase_id),2
       ) as "Average Revenue per Purchase"
from Courses c
Inner join purchases p
    on c.course_id = p.course_id
Group by c.category;
```

Key Insights:

- Total revenue generated: ₹1,31,000
- Jing is the highest-spending learner
- Global Health Policy generates the highest revenue
- Medicine is the highest-revenue category
- Chinese for Beginners has strong purchase quantity but lower revenue
- Master of Data Science has strong revenue performance
- Learners are purchasing across multiple categories

Overall Business Conclusion:

The learner project generated ₹1.31 lakh in revenue from 8 transactions and 13 course units. Jing is the highest-spending learner, while Global Health Policy is the highest-revenue course and Medicine is the strongest-performing category. Higher-priced courses such as Global Health Policy and Master of Data Science contribute disproportionately to total revenue, demonstrating that revenue performance depends

on both purchase volume and course pricing. The data also indicates opportunities for cross-selling across categories. However, country-level spending comparisons are currently limited because each country has only one learner.